

C++ Module 09

Guide Complet de la STL

Conteneurs · Itérateurs · Algorithmes · Ford-Johnson

C++98 Standard

Table des matières

N°	Chapitre
1	Rappels C++98 et Forme Canonique
2	std::map — Arbre Rouge-Noir
3	std::stack — Adaptateur LIFO
4	std::vector — Tableau Dynamique
5	std::deque — File Double Extrémité
6	Itérateurs
7	Algorithmes STL

8	Gestion des exceptions
9	Algorithme Ford-Johnson
10	Complexité algorithmique
11	Récapitulatif des erreurs courantes

1 · Rappels C++98 et Forme Canonique Orthodoxe

La Forme Canonique Orthodoxe (Orthodox Canonical Form)

Depuis le Module 02, toute classe C++ doit implémenter ces quatre méthodes spéciales. Leur absence force le compilateur à en générer des versions par défaut qui effectuent une copie superficielle (shallow copy) — dangereuse si la classe possède des ressources allouées dynamiquement.

```
class MaClasse
{
public:
    MaClasse(); // 1. Constructeur par défaut
    MaClasse(const MaClasse& other); // 2. Constructeur de copie
    MaClasse& operator=(const MaClasse& other); // 3. Opérateur d'affectation
    ~MaClasse(); // 4. Destructeur

private:
    int* _data; // ressource gérée manuellement
    size_t _size;
};

// Implémentation correcte (deep copy)
MaClasse::MaClasse(const MaClasse& other)
    : _size(other._size)
{
    _data = new int[_size];
    for (size_t i = 0; i < _size; ++i)
        _data[i] = other._data[i];
}

MaClasse& MaClasse::operator=(const MaClasse& other)
{
    if (this != &other) // Protection contre a = a
    {
        delete[] _data;
        _size = other._size;
        _data = new int[_size];
        for (size_t i = 0; i < _size; ++i)
            _data[i] = other._data[i];
    }
    return *this;
}
```

Pourquoi this != &other; ?

Sans cette garde, l'auto-affectation (a = a) supprimerait les données AVANT de les recopier. Résultat : données perdues et comportement indéfini.

Include Guards

Empêchent l'inclusion multiple d'un même header. Sans eux, deux fichiers .cpp incluant le même header provoqueraient des erreurs de redéfinition.

```
#ifndef MON_HEADER_HPP
# define MON_HEADER_HPP

// Déclarations...

#endif // MON_HEADER_HPP
```

Restrictions C++98 importantes dans ce module

Interdit	Autorisé / Alternative
C++11 et supérieur	C++98 uniquement (-std=c++98)
using namespace std;	Préfixer : std::vector, std::map...
friend keyword	Interfaces publiques bien définies
*printf(), *alloc(), free()	std::cout, new/delete, RAII
Boost libraries	STL standard uniquement
auto, lambda, range-for	Types explicites, boucles classiques

2 · std::map — Arbre Rouge-Noir (ex00)

Qu'est-ce que std::map ?

std::map est un conteneur associatif ordonné. Chaque élément est une paire (clé, valeur). Les clés sont TOUJOURS triées en ordre croissant grâce à un comparateur (par défaut std::less). En interne, c'est un arbre binaire de recherche auto-équilibré (arbre rouge-noir).

Propriété	Valeur	Détail
Structure interne	Arbre rouge-noir	BST auto-équilibré
Accès par clé []	O(log n)	Crée si absente !
find()	O(log n)	Retourne end() si absent
lower_bound()	O(log n)	Premier element >= clé
upper_bound()	O(log n)	Premier element > clé
insert()	O(log n)	Ne remplace PAS si existe
erase()	O(log n)	Supprime par clé ou itérateur
Itération	O(n)	Toujours en ordre croissant
Clés uniques	Oui	std::multimap pour doublons

Utilisation de base

```
#include <map>
#include <string>
#include <iostream>

// Déclaration : map<TypeClé, TypeValeur>
std::map<std::string, double> btcPrices;

// Insertion – 3 façons équivalentes
btcPrices["2011-01-03"] = 0.30;           // operator[] : crée si absent
btcPrices.insert(std::make_pair("2012-01-11", 7.10)); // insert()

// Itération (toujours en ordre des clés)
std::map<std::string, double>::iterator it;
for (it = btcPrices.begin(); it != btcPrices.end(); ++it)
    std::cout << it->first << " : " << it->second << std::endl;

// Recherche
it = btcPrices.find("2011-01-03");
if (it != btcPrices.end())
    std::cout << "Trouvé : " << it->second << std::endl;
```

lower_bound() — Le coeur de ex00

`lower_bound(key)` retourne un itérateur vers le PREMIER élément dont la clé est $\geq$ `key`. C'est l'opération clé pour trouver la date la plus proche dans la base de données Bitcoin.

```
// Exemple : trouver le taux pour une date donnée (ou la plus proche inférieure)
std::map<std::string, double>::const_iterator it = _database.lower_bound(date);

// Cas 1 : it == end() → date > toutes les dates → reculer d'un pas
// Cas 2 : it->first == date → correspondance exacte
// Cas 3 : it->first > date → reculer d'un pas pour avoir la date <= recherchée
// Cas 4 : it == begin() → toutes les dates sont > date → erreur

if (it == _database.end() || it->first != date)
{
    if (it == _database.begin())
        // Erreur : aucune date <= date dans la DB
    else
        --it; // Reculer vers la clé précédente (la plus proche <=)
}
// Maintenant it->first est la date la plus proche <=
double rate = it->second;

// SCHÉMA :
// DB : [ "2010-01-01" | "2011-06-15" | "2012-03-20" ]
// Chercher "2011-09-01"
// lower_bound → itérateur sur "2012-03-20" (première clé >= "2011-09-01")
// --it → itérateur sur "2011-06-15" (la plus proche <=) ✓
```

Astuce ISO 8601 : Les dates au format YYYY-MM-DD ont la propriété que leur ordre lexicographique (alphabétique) est identique à leur ordre chronologique. C'est pourquoi `std::map<std::string, double>` avec des clés en format ISO 8601 trie automatiquement les dates dans le bon ordre !

3 · std::stack — Adaptateur LIFO (ex01)

Adaptateur de conteneur vs. Conteneur

std::stack n'est PAS un conteneur — c'est un adaptateur de conteneur. Il enveloppe un conteneur sous-jacent (std::deque par défaut) et restreint son interface pour n'exposer que les opérations LIFO. Cette restriction est intentionnelle : elle rend le code auto-documenté et prévient les erreurs.

Méthode	Complexité	Description
push(val)	O(1)	Empile val au sommet
pop()	O(1)	Retire le sommet (ne retourne RIEN)
top()	O(1)	Accède au sommet sans le retirer
size()	O(1)	Nombre d'éléments
empty()	O(1)	Vrai si la pile est vide

LIFO — Last In, First Out

LIFO signifie que le dernier élément empilé est le premier à être dépilé. C'est exactement le comportement nécessaire pour évaluer une expression RPN : on empile les opérandes, et quand on rencontre un opérateur, on dépile les deux derniers pour calculer.

```
// Algorithme RPN : "8 9 * 9 - 4 - 1 +"
// Trace d'exécution :

std::stack<int> stack;

// Token "8" → push(8)      pile: [8]
// Token "9" → push(9)      pile: [8, 9]    ← sommet = 9
// Token "*" → right=pop()=9, left=pop()=8
//              push(8*9=72) pile: [72]
// Token "9" → push(9)      pile: [72, 9]
// Token "-" → right=9, left=72
//              push(72-9=63) pile: [63]
// Token "4" → push(4)      pile: [63, 4]
// Token "-" → right=4, left=63
//              push(63-4=59) pile: [59]
// Token "1" → push(1)      pile: [59, 1]
// Token "+" → right=1, left=59
//              push(59+1=60) pile: [60]
// Fin : stack.size() == 1, résultat = stack.top() = 60

// ATTENTION : l'ordre des opérandes pour - et / !
int right = stack.top(); stack.pop(); // Dépiler d'abord le right
int left  = stack.top(); stack.pop(); // Puis le left
// Calculer : left OP right (et NON right OP left !)
stack.push(left - right); // left - right ≠ right - left !
```

std::istringstream — Tokeniser une string

Pour découper l'expression RPN en tokens, on utilise std::istringstream qui permet de lire une string comme un flux, avec l'opérateur >>.

```
#include <sstream>

std::string expression = "8 9 * 9 -";
std::istringstream iss(expression);
std::string token;

while (iss >> token)           // >> lit un mot (séparé par espaces)
{
    std::cout << token << std::endl;
    // Affiche : "8", "9", "*", "9", "-"
}
```


4 · std::vector — Tableau Dynamique (ex02)

Structure interne

std::vector stocke ses éléments dans un bloc de mémoire CONTIGUË, exactement comme un tableau C. Quand le tableau est plein et qu'on ajoute un élément, vector alloue un nouveau bloc (typiquement 2x plus grand), copie tous les éléments, et libère l'ancien bloc.

Opération	Complexité	Remarque
operator[i] / at(i)	O(1)	Accès direct par indice
push_back()	O(1) amorti	Peut déclencher une réallocation
pop_back()	O(1)	Retire le dernier élément
insert(pos, val)	O(n)	Décale tous les éléments suivants
erase(pos)	O(n)	Décale tous les éléments suivants
size()	O(1)	Nombre d'éléments actuels
reserve(n)	O(n)	Pré-alloue n emplacements
begin() / end()	O(1)	Itérateurs

Localité de cache : Comme std::vector est contigu en mémoire, le CPU peut charger plusieurs éléments en cache en une seule opération (cache line = 64 octets, soit 16 int). C'est pourquoi vector est généralement plus rapide que deque ou list pour les accès séquentiels.

```
#include <vector>

std::vector<int> v;

v.push_back(3);           // [3]
v.push_back(1);           // [3, 1]
v.push_back(4);           // [3, 1, 4]

// Accès par indice (O(1))
std::cout << v[0];        // 3
std::cout << v.size();    // 3

// Insertion en milieu (O(n) : déplace les éléments)
v.insert(v.begin() + 1, 99); // [3, 99, 1, 4]

// Itération avec itérateur
std::vector<int>::iterator it;
for (it = v.begin(); it != v.end(); ++it)
    std::cout << *it << " ";

// Construction depuis un autre vector (copie profonde)
std::vector<int> copy = v;    // Copie complète
```

5 · std::deque — File Double Extrémité (ex02)

Structure interne — blocs non contigus

Contrairement à `std::vector`, `std::deque` n'utilise PAS un bloc de mémoire unique et contigu. Elle alloue la mémoire par blocs de taille fixe, et maintient un tableau de pointeurs vers ces blocs. Chaque accès `deque[i]` nécessite donc une double indirection : trouver le bon bloc, puis l'élément dans ce bloc.

Opération	Complexité	vs vector
<code>operator[i]</code>	$O(1)$	Plus lent (double indirection)
<code>push_back()</code>	$O(1)$	Similaire
<code>push_front()</code>	$O(1)$	AVANTAGE : $O(1)$ vs $O(n)$ pour vector
<code>pop_front()</code>	$O(1)$	AVANTAGE : $O(1)$ vs $O(n)$ pour vector
<code>insert(milieu)</code>	$O(n)$	Similaire
Mémoire	Fragmentée	DÉSAVANTAGE : mauvais cache locality

Comparaison des performances

```
// Sur 3000 éléments, résultats typiques observés :
// std::vector : ~1000 us (meilleur cache locality)
// std::deque : ~3000 us (fragmentation mémoire)
// → std::vector est 2-5x plus rapide sur cet algorithme

// Quand préférer std::deque ?
// → Insertions/suppressions fréquentes aux DEUX extrémités
// → File d'attente (queue) avec push_back + pop_front
// → Taille imprévisible : pas de réallocation massive comme vector

// Interface std::deque (identique à vector + push/pop_front)
std::deque<int> d;
d.push_back(3);    // [3]
d.push_front(1);   // [1, 3] ← push_front = O(1) !
d.push_back(4);    // [1, 3, 4]
d.pop_front();     // [3, 4] ← pop_front = O(1) !
```

6 · Itérateurs — Naviguer dans les Conteneurs

Concept fondamental

Un itérateur est un objet qui permet de parcourir un conteneur sans connaître son implémentation interne. C'est le pont entre les algorithmes et les conteneurs dans la STL. On peut voir un itérateur comme un pointeur généralisé : il supporte les opérations `*`, `->`, `++`, `--`, `==` et `!=`.

Catégorie	Opérations	Exemples
Input	<code>++</code> , <code>*</code> , <code>==</code> , <code>!=</code>	<code>std::istream_iterator</code>
Forward	<code>++</code> , <code>*</code> , <code>==</code> , <code>!=</code>	<code>std::forward_list</code>
Bidirectionnel	<code>++</code> , <code>--</code> , <code>*</code> , <code>==</code> , <code>!=</code>	<code>std::list</code> , <code>std::map</code> , <code>std::set</code>
Random Access	<code>++</code> , <code>--</code> , <code>+n</code> , <code>-n</code> , <code>[]</code> , <code>==</code> , <code><</code>	<code>std::vector</code> , <code>std::deque</code>

```
// Itérateur de std::map (bidirectionnel)
// it->first = clé, it->second = valeur
std::map<std::string, double>::iterator it;
for (it = myMap.begin(); it != myMap.end(); ++it)
{
    std::cout << it->first << " -> " << it->second << std::endl;
}

// const_iterator : lecture seule (utiliser quand méthode const)
std::map<std::string, double>::const_iterator cit = myMap.begin();

// Reculer d'un pas (bidirectionnel)
std::map<std::string, double>::iterator last = myMap.end();
--last; // Dernier élément

// Itérateur de std::vector (random access)
std::vector<int>::iterator vit;
vit = vec.begin() + 3; // Sauter directement à l'indice 3 (0(1))
std::cout << *vit;    // Déréférencement

// Itérateur invalide : éviter de stocker un itérateur après insertion !
// vec.insert(...) peut invalider TOUS les itérateurs de vec
// map.insert(...) n'invalide PAS les itérateurs existants
```

7 · Algorithmes STL —

Principe

Les algorithmes STL opèrent sur des RANGES d'itérateurs [begin, end), pas directement sur des conteneurs. Cela les rend universels : le même algorithme fonctionne avec vector, list, deque, etc. Ce module (09) autorise enfin l'utilisation de .

```
#include <algorithm>

std::vector<int> v = {5, 3, 1, 4, 2};

// std::sort : O(n log n), algorithme introsort (quicksort + heapsort)
std::sort(v.begin(), v.end());           // [1, 2, 3, 4, 5]
std::sort(v.begin(), v.end(), std::greater<int>()); // [5, 4, 3, 2, 1]

// std::binary_search : O(log n), nécessite tableau trié
bool found = std::binary_search(v.begin(), v.end(), 3); // true

// std::lower_bound / upper_bound : O(log n)
std::vector<int>::iterator lb = std::lower_bound(v.begin(), v.end(), 3);
// lb pointe vers le premier element >= 3

// std::find : O(n), recherche linéaire
std::vector<int>::iterator it = std::find(v.begin(), v.end(), 4);

// std::swap : échange deux valeurs, O(1)
std::swap(v[0], v[4]);

// std::reverse : inverse in-place, O(n)
std::reverse(v.begin(), v.end());

// std::min / std::max
int smallest = std::min(3, 7); // 3
```

8 · Gestion des Exceptions en C++98

try / catch / throw

Les exceptions permettent de séparer le code de traitement normal du code de gestion d'erreurs. Contrairement aux codes de retour, les exceptions remontent automatiquement la pile d'appel jusqu'au premier catch correspondant.

```
#include <stdexcept> // std::runtime_error, std::invalid_argument...
#include <exception>  // std::exception (classe de base)

// Lancer une exception
void divide(int a, int b)
{
    if (b == 0)
        throw std::runtime_error("Division par zéro !");
    return a / b;
}

// Interceptor une exception
int main()
{
    try
    {
        int result = divide(10, 0); // Lance une exception
        std::cout << result;       // Ne sera jamais exécuté
    }
    catch (const std::runtime_error& e)
    {
        std::cerr << "Erreur : " << e.what() << std::endl;
    }
    catch (const std::exception& e) // Catch générique (tous les std::exception)
    {
        std::cerr << "Exception : " << e.what() << std::endl;
    }
    return 0;
}

// Hiérarchie des exceptions standard :
// std::exception
//   ■■■ std::logic_error
//   ■   ■■■ std::invalid_argument
//   ■   ■■■ std::out_of_range
//   ■   ■■■ std::length_error
//   ■■■ std::runtime_error
//       ■■■ std::overflow_error
//       ■■■ std::underflow_error
```

9 · Algorithme Ford-Johnson (Merge-Insert Sort)

Contexte historique

L'algorithme Ford-Johnson (1959) est le premier algorithme de tri connu à être optimal en nombre de comparaisons pour certaines valeurs de n . Il est décrit dans TAOCP Vol.3, Knuth, p.184. La borne inférieure théorique est $\log_2(n!)$ comparaisons (nombre minimum pour distinguer $n!$ permutations). Ford-Johnson l'atteint pour $n \leq 11$ et reste très proche pour les grands n .

Les 3 phases de l'algorithme

Phase 1 — Trier les paires

```
// Séquence : [3, 1, 4, 1, 5, 9, 2, 6]
// On parcourt par paires et on trie chaque paire :
// (3,1) → (1,3)   (4,1) → (1,4)   (5,9) → (5,9)   (2,6) → (2,6)
//
// Résultat : paires = [(1,3), (1,4), (5,9), (2,6)]
//             pend  = [1, 1, 5, 2]   (les petits)
//             main   = [3, 4, 9, 6]   (les grands)
//
// 1 comparaison par paire = n/2 comparaisons totales pour cette phase
```

Phase 2 — Trier récursivement la main chain

```
// On trie récursivement les "grands" par Ford-Johnson :
// main = [3, 4, 9, 6] → après tri récursif → [3, 4, 6, 9]
//
// IMPORTANT : les pends restent couplés à leurs grands !
// Après tri : paires triées = [(1,3), (1,4), (2,6), (5,9)]
//             pends (dans l'ordre de main triée) = [1, 1, 2, 5]
//
// La main chain triée sera la base dans laquelle on va insérer les pends
```

Phase 3 — Insérer les pends (ordre de Jacobsthal)

```
// main chain initiale : [3, 4, 6, 9]
// pends (dans l'ordre) : [p0=1, p1=1, p2=2, p3=5]
//
// Étape 3a : Insérer p0=1 en tête (garanti <= main[0]=3)
// main : [1, 3, 4, 6, 9]
//
// Étape 3b : Insérer les pends restants en ordre de Jacobsthal
// Groupes Jacobsthal : [1, 3, 5, 11, 21, ...]
// Pour 3 pends restants : groupes = [1, 3] → (p1), puis (p3, p2)
//
//   Insérer p1=1 : binary search dans [1,3,4,6,9] → position 1
//   main : [1, 1, 3, 4, 6, 9]
//
//   Insérer p3=5 : binary search dans [1,1,3,4,6,9] → position 4
//   main : [1, 1, 3, 4, 5, 6, 9]
//
//   Insérer p2=2 : binary search dans [1,1,3,4,5,6,9] → position 2
//   main : [1, 1, 2, 3, 4, 5, 6, 9] ← Résultat final trié !
```

Les nombres de Jacobsthal

La suite de Jacobsthal dicte l'ordre d'insertion des pends pour minimiser le nombre de comparaisons de la recherche binaire.

```
// Formule :  $J(0)=0$ ,  $J(1)=1$ ,  $J(n) = J(n-1) + 2*J(n-2)$ 
// Séquence : 0, 1, 1, 3, 5, 11, 21, 43, 85, 171, ...
//
// "Fins de groupes" utilisées pour l'insertion :
//   1, 3, 5, 11, 21, 43, 85, ...
//   avec formule :  $t(1)=1$ ,  $t(2)=3$ ,  $t(k) = t(k-1) + 2*t(k-2)$ 
//
// POURQUOI CET ORDRE ?
// Quand on insère le pend à l'indice k dans un groupe,
// la main chain contient exactement  $2^j$  éléments dans la zone de recherche.
// binary search dans  $2^j$  elements = exactement j comparaisons = OPTIMAL
//
// Sans Jacobsthal (ordre naïf 1,2,3,4,...) :
//   Insérer p2 : borne = position de main[1] = 2 →  $\log_2(2) = 1$  comparaison
//   Insérer p3 : borne = position de main[2] = 4 →  $\log_2(4) = 2$  comparaisons
//
// Avec Jacobsthal (ordre p1, p3, p2, ...) :
//   Insérer p3 d'abord : borne = position de main[2] = 4 → 2 comparaisons
//   Insérer p2 ensuite : borne = position de main[1] qui est déjà fixé
//   Résultat : même nombre ou moins de comparaisons au total
```


10 · Complexité Algorithmique — Notation Big-O

Définition et intuition

La notation Big-O décrit comment le temps d'exécution (ou l'espace mémoire) d'un algorithme évolue en fonction de la taille de l'entrée n , dans le pire cas. On ignore les constantes et les termes dominés.

Notation	Nom	Exemple	$n=1000$
$O(1)$	Constante	Accès <code>map[]</code>	1 op
$O(\log n)$	Logarithmique	<code>lower_bound</code> , binary search	~10 op
$O(n)$	Linéaire	Parcours d'un tableau	1 000 op
$O(n \log n)$	Quasilinéaire	<code>std::sort</code> , Ford-Johnson	~10 000 op
$O(n^2)$	Quadratique	Tri bulle, insertion naïve	1 000 000 op
$O(2^n)$	Exponentielle	Backtracking sans optimisation	2^{1000} op !

Magie du logarithme : Pour $n = 1\,000\,000\,000$ éléments, $\log_2(n) \approx 30$. Une recherche binaire dans un milliard d'éléments triés prend au maximum 30 comparaisons. C'est pourquoi `std::map` avec `lower_bound` est parfait pour chercher parmi des années de données Bitcoin.

11 · Erreurs Courantes et Comment les Éviter

Déréférencer end()

Problème : `end()` n'est PAS un élément valide, c'est une sentinelle.

Solution : Toujours vérifier `it != container.end()` avant `*it` ou `it->`

`operator[]` sur map crée la clé

Problème : `myMap["clé"]` crée l'entrée avec valeur 0 si absente.

Solution : Utiliser `find()` pour vérifier l'existence sans créer.

Invalidation d'itérateurs (vector)

Problème : `push_back` ou `insert` peuvent déclencher une réallocation.

Solution : Ne pas stocker d'itérateurs vector à travers des insertions.

`pop()` de stack vide

Problème : `stack::pop()` sur pile vide = comportement indéfini.

Solution : Toujours vérifier `!stack.empty()` avant `pop()`.

Ordre des opérandes (RPN)

Problème : Pour `a - b`, `right = top()` puis `left = top()`. Inverser = bug.

Solution : `right = top(); pop(); left = top(); pop(); résultat = left OP right.`

Auto-affectation

Problème : `a = a` sans garde supprime les données avant de les copier.

Solution : Toujours tester `if (this != &other;)` dans `operator=`.

`strtod` vs `atof`

Problème : `atof` ne détecte pas les conversions partielles ("`3.5abc`").

Solution : Utiliser `strtod` avec `endPtr` pour valider la conversion entière.

`std::pair` et indices

Problème : `pairs[i].first` = petit, `pairs[i].second` = grand.

Solution : Bien documenter la convention pour ne pas confondre.

Récapitulatif — Conteneurs du Module 09

Exercice	Conteneur	Raison du choix	Opération clé
ex00 Bitcoin	std::map	Clés triées, recherche efficace par date	lower_bound()
ex01 RPN	std::stack	Sémantique LIFO native, interface minimale	push/pop/top
ex02 PmergeMe	std::vector	Mémoire contigüe, accès rapide par indice	insert + binary search
ex02 PmergeMe	std::deque	Même algo, perf différente (comparaison)	insert + binary search